ghostly-runs: Optimizing and Benchmarking ghostly-neighbors for Neighborlist Construction

Michael Almeida **5/7/2026**
mjalmeida172@gmail.com cs2050

Introduction

Molecular dynamics simulations spend most of their wall clock inside one tiny subroutine: **neighbor list construction**. Given *N* atoms and a cutoff radius *r_c*, find every pair *(i, j)* with distance *d(i,j) < r_c*. The naïve all-pairs version is O(N²); the standard fix is a **cell-list**, which partitions space into cells of side ≥ *r_c* and reduces the search to the 27 neighboring cells per atom — expected O(N).

In this report, I take this one algorithm and walk it through five parallelization regimes: serial C++ with hand-rolled AVX-512 intrinsics, OpenMP shared-memory threading, MPI distributed-memory decomposition, CUDA dispatched via CuPy/NVRTC with CUDA C++ written kernels , and a "productivity language" comparison using Numba, JAX, and Kokkos. The serial, OpenMP, CUDA, and productivity-framework backends were authored for this report; the MPI domain decomposition was provided in the project scaffold and exercised here, not implemented. The Numba and JAX backends were similarly not authored for this report, but exercised here. All backends are verified against a common reference and benchmarked on the Harvard HUIT cluster.

> A more detailed companion document with full Nsight progression, multi-node debugging notes, and code snippets is available in `report-full-annotated.pdf`

The algorithm

Four stages, used in every backend:

1. **Cell assignment.** Each atom maps to a 3D grid cell of side length *r_c*.
2. **Cell-list construction.** Histogram → scan → scatter type algorithm to construct the stencil.
3. **Pair search.** For each owned atom *i*, walk the 3×3×3 cell stencil, compute squared distances, accept pairs below r_c^2.
4. **Output.** Store *(i, j)* and *d_ij*.

CPU side: AVX-512, OpenMP, and the perf-driven two-pass

The serial baseline is a brute-force O(N²) loop and comes in two flavors: generic and SIMD. On it's own, the SIMD version is a flat ~2x performance improvement.

The cell-list version threaded with OpenMP is where it gets interesting. The first attempt was a single-pass kernel: each thread grew a `std::vector` of pairs as it walked the stencil. This `perf` profile is what the kernel looked like before:

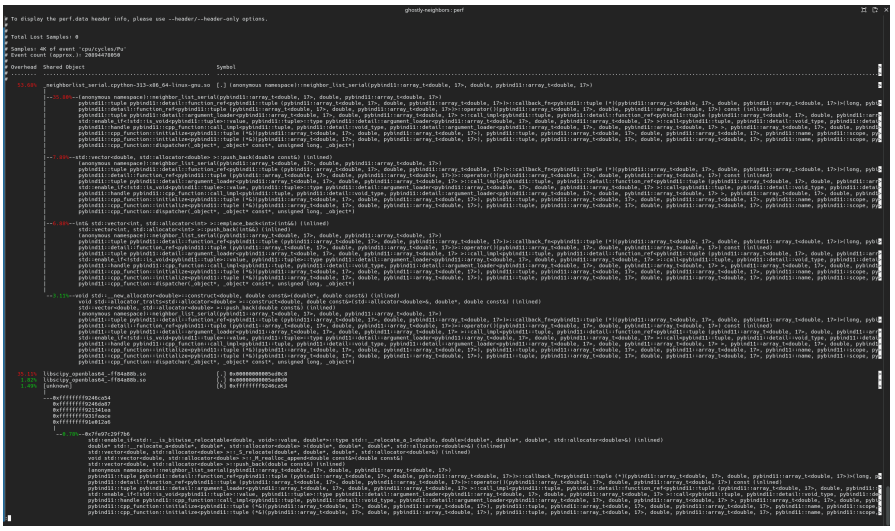


Figure 1a: 53.7% of cycles inside `neighbor_list_serial`, dominated by `std::vector::push_back` reallocation paths (~35% `push_back`, ~7% `emplace_back`, ~6% `_M_realloc_insert`).

The kernel was spending most of its time **moving memory around**, not computing distances. The working point showed 108k atoms at 12 Å cutoff → ~68.8M pairs × 32 B = 2.2 GB final storage, so `std::vector`'s doubling-realloc strategy moved roughly 2× that — ~4.4 GB shuffled per neighbor-list construction, every MD step.

The fix is a **two-pass kernel**: pass 1 calculates counts per atom and a subsequent serial prefix sum produces exact `offsets[i]`, pass 2 walks the same cells and *compress-stores* into pre-allocated NumPy arrays at known offsets. This elimates the reallocations that were thrashing the cache performance. The serial prefix sum is an unexplored optimzation opportunity.

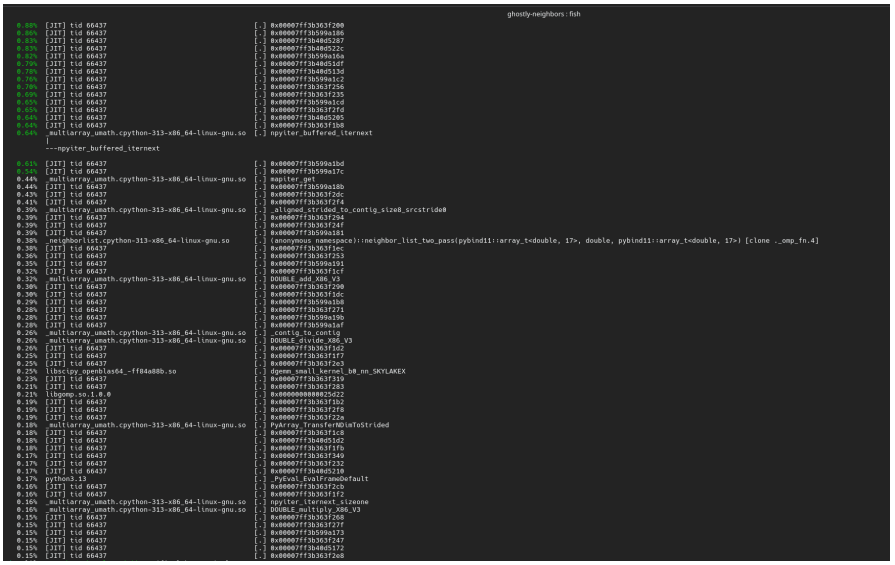


*Figure 1b: Same simulation, two-pass kernel. The neighbor list is now **0.38%** of cycles. BLAS and the Numba-JIT LJ potential dominate. ~140× reduction in the kernel's share of runtime.*

Moving to a two pass algorithm, despite the percieved wastefulness of an additional pass, contains the entire optimization story for this algorithm-- minimize memory movement.

CPU benchmark and OpenMP scaling

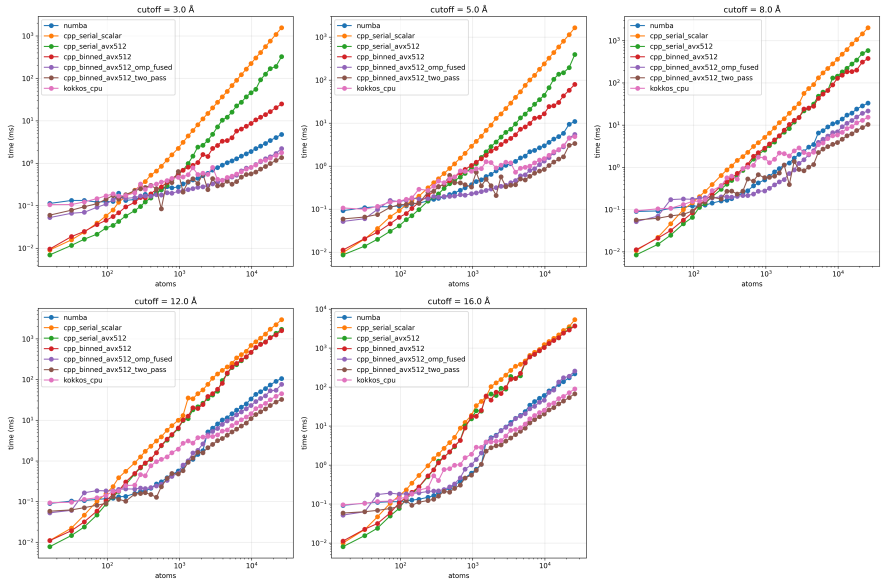
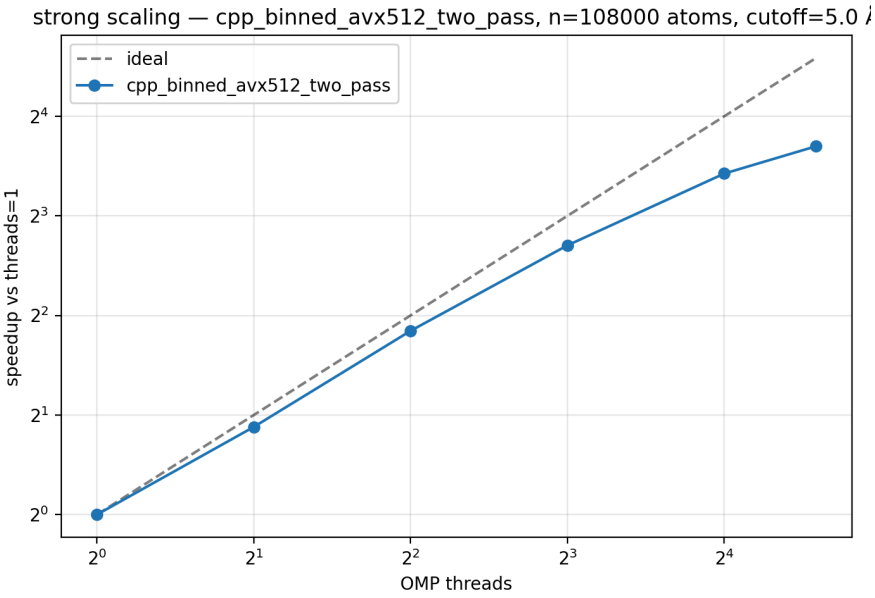


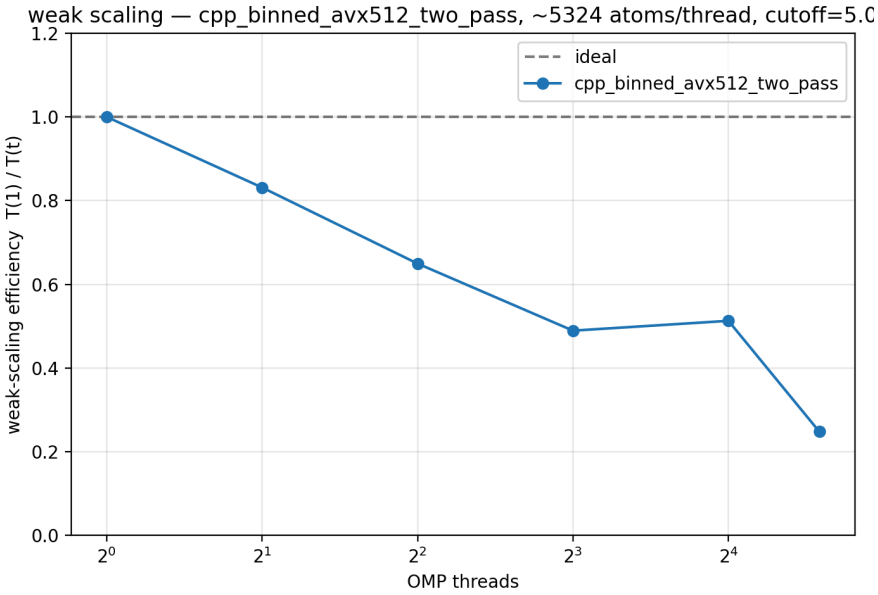
Figure 2: Runtime vs. atom count for all CPU backends at five cutoffs. The naïve O(N²) kernels diverge from the cell-list ones beyond ~1000 atoms; Kokkos has ~7 ms constant overhead but matches at large N; Numba is competitive at small N and falls behind at large N due to less aggressive SIMD utilization.

Strong-scaling on 108k atoms:



Threads	Time (ms)	Speedup	Efficiency
1	201.7	1.00×	100%
8	31.0	6.51×	81%
16	18.8	10.74×	67%
24	15.5	**12.98×**	54%

The cliff above 16 threads is DRAM bandwidth saturation on the single socket. Critically, parallelizing the **cell-list build phase** (not just the pair search) was what made this work — without it, Amdahl's law capped speedup at ~1.13×. Weak scaling tells the same story:



83% efficiency at 2 threads, 65% at 4, 49% at 8, 51% at 16, 25% at 24. Mostly clean through 16, then communication and prefix-sum costs catch up.

GPU side: same same, but different stalls

The CUDA kernel is dispatched via CuPy's `RawModule` (NVRTC) — point at a `.cu` file, get a compiled kernel. The advantage of CuPy is iteration speed; the wrapper looks like NumPy code with a few `.get_function(...)` calls and the prefix-scan step is just `cp.cumsum` (CUB under the hood).

The kernel went through three iterations, each driven by Nsight Compute:

- **Naive (one thread per atom).** SM throughput 53.7%, IPC 0.42, 57% uncoalesced sectors, **84% of warp cycles stalled at CTA barriers**. Latency-bound.
- **Tiled with shared memory.** Loaded cell segments into shared tiles before the distance check. Memory throughput up to 23.8%, uncoalesced down to 35% — but barriers still 84% of stall cycles, because warps idled at `__syncthreads()` waiting for slow lanes. Net win was modest.
- **Warp-level primitives.** Replaced `__syncthreads()` with warp shuffles and ballot intrinsics. One warp per atom (32 lanes), one global atomic per warp instead of one per thread. Walkthrough in the appendix.

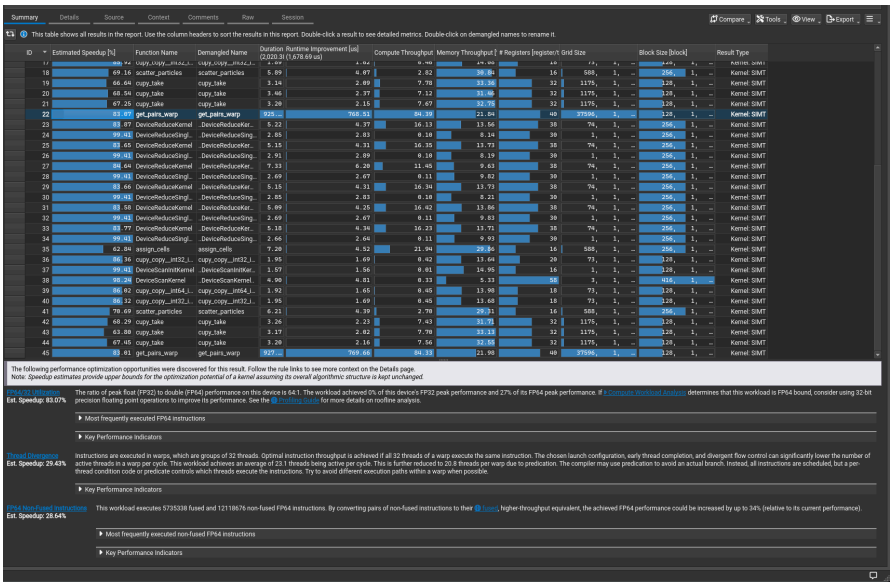


*Figure 3: Nsight Compute summary — `get_pairs_warp` lands at **84.39% compute throughput** (Nsight reclassifies it from "Latency Issue" to "High Throughput"), IPC 0.82, uncoalesced down to 19%. The remaining bottleneck is the FP64 pipeline at 84% utilization, not memory or synchronization.*

Metric	Naive	Tiled	Warp
SM throughput	53.7%	56.0%	**84.3%**
Memory throughput	10.4%	23.8%	22.0%
Uncoalesced sectors	57%	35%	**19%**
Top bottleneck	Barrier stalls	Barrier stalls	**FP64 pipeline**
Nsight class	Latency Issue	Latency Issue	**High Throughput**

The arc mirrors the CPU side step-for-step. CPU optimization removed allocator overhead via prefix-summed two-pass writes; GPU optimization removed barrier stalls via warp shuffles. In both cases the headline win came from **eliminating a non-arithmetic stall**, not from finding more FLOPs.

It's important to note that the cards on the HUIT Cluster (L4's) have nerfed FP64 performance, which comprises the core of the GPU kernels. Follow on work would see this implemented on an enterprise class card where full Fp64 performance is available.

GPU benchmark

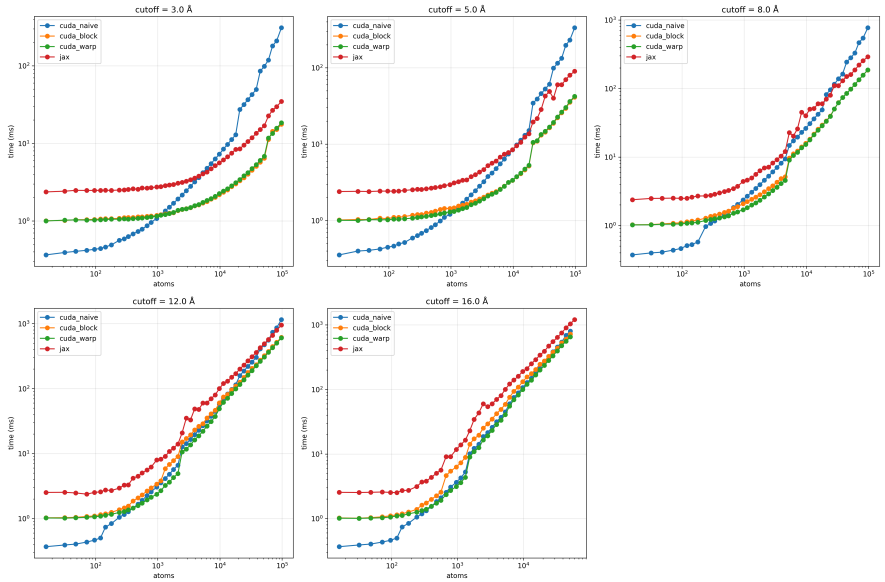


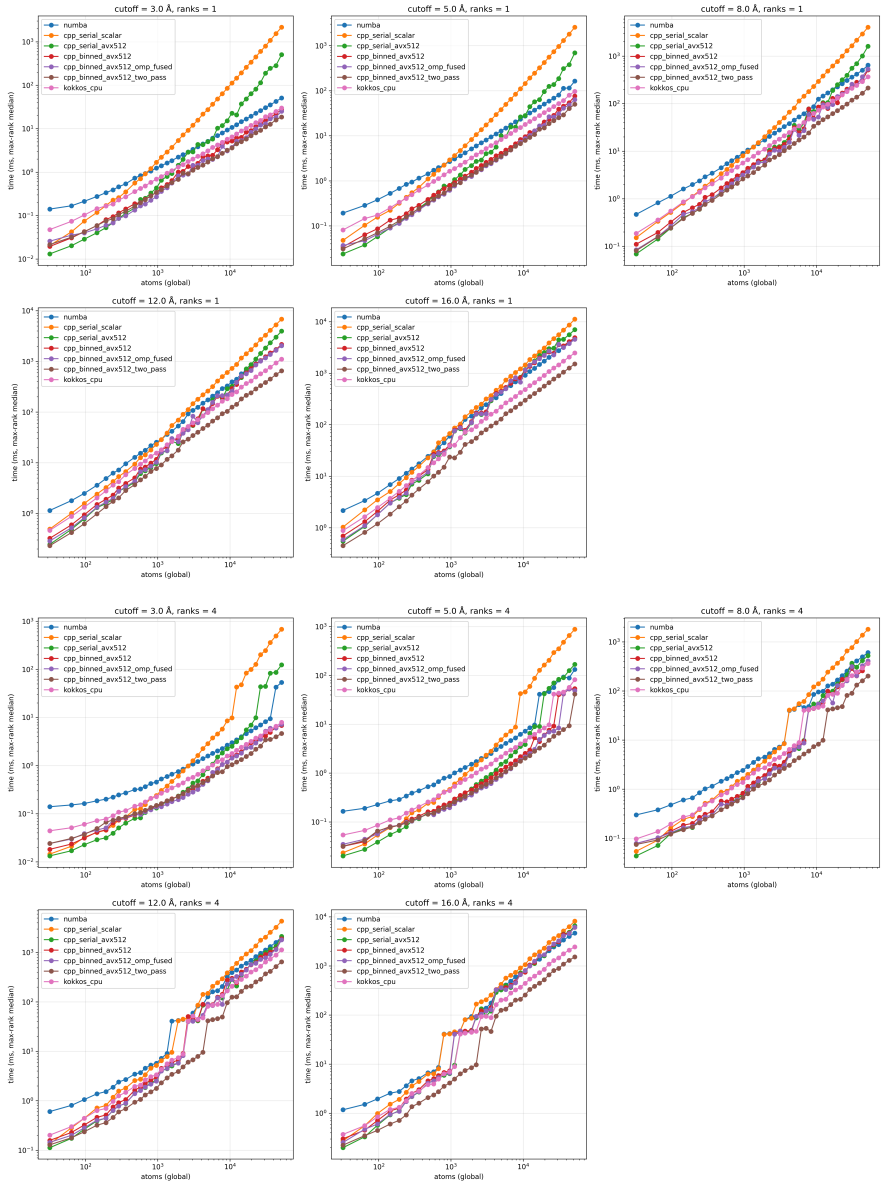
Figure 4: All single-GPU backends at five cutoffs on the L4. `cuda_block` and `cuda_warp` track each other through most of the sweep; `cuda_naive` lags at small/medium N and catches up at large N as work hides its uncoalesced gathers. JAX is competitive below ~1k atoms then falls off because XLA isn't producing a maximally specialized kernel. `kokkos_gpu` is missing here — see Framework Comparison for the OOM story.

MPI scaling

The provided `DistributedParticles` class uses a 3D Cartesian topology (`MPI.Compute_dims(comm.size, 3)` + periodic `Create_cart`), exchanges ghost atoms across the 26 face/edge/corner neighbors, and accumulates ghost forces back to owners. I built the benchmarking harness around it and ran it across the full grid at 1/2/4 ranks single-node, then 48/96/192 ranks across multiple nodes.

Note For project completion, a C++ MPI implementation, containing a simple 2D slab exchange is included. It's not benchmarched in this report, but it is contained in the `mpi` directory, with an sbatch script to run the example to verify correctness.

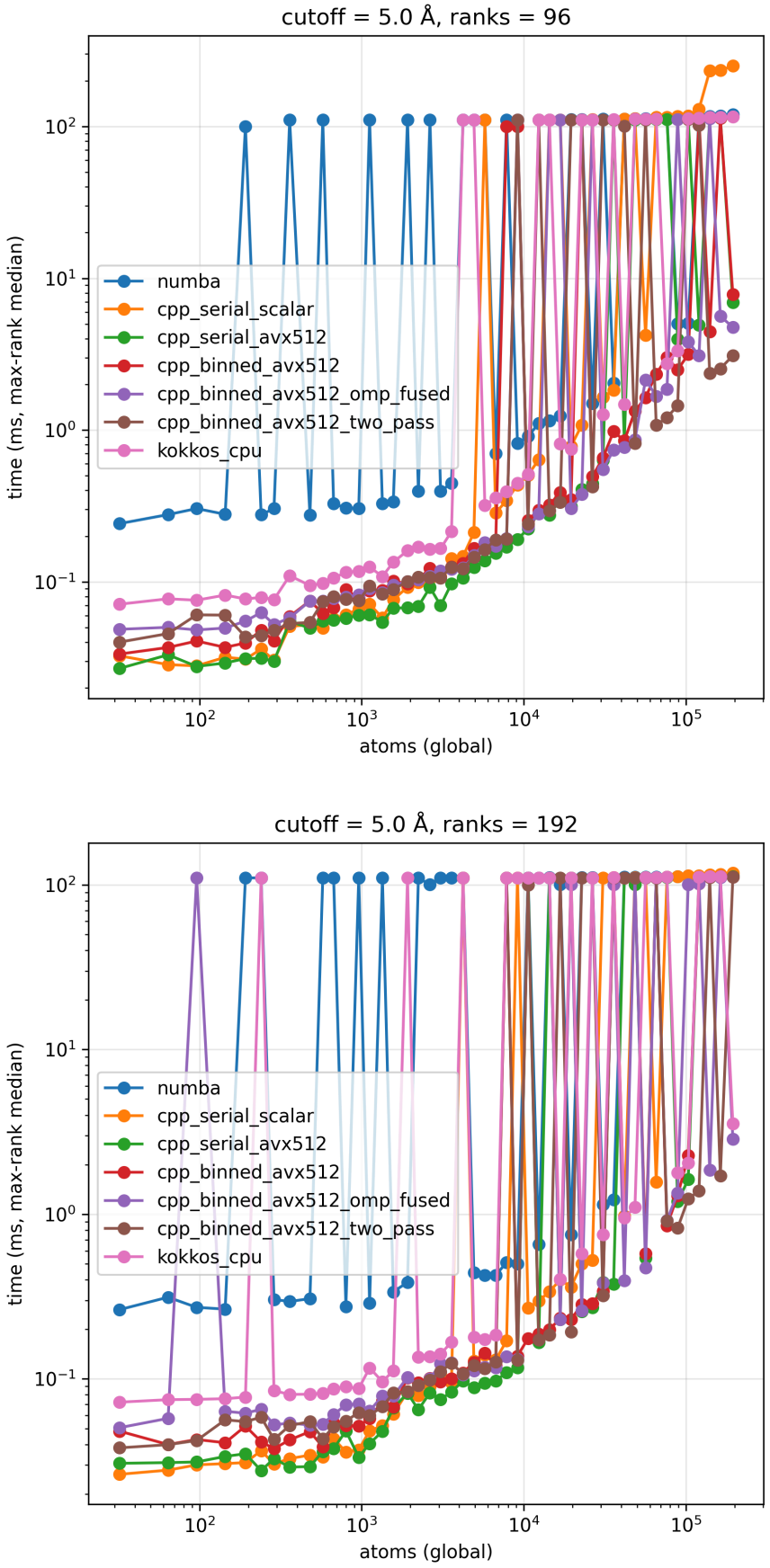
CPU Intranode MPI: clean scaling, then bandwidth saturation



Figures 5a–b: CPU MPI sweeps at 1 and 4 ranks (single node). The cell-list curves slide down by ~2× per rank doubling. At 22,800 atoms, 4 ranks hit 3.98× speedup (99% efficiency). Below ~1k atoms the 4-rank curve is communication-bound; at 51,984 atoms it collapses to 1.21× as DRAM bandwidth saturates across the 4 cores.

CPU Multinode MPI: real gains under HPC noise

Pushing past one node, we ran 48 / 96 / 192 ranks (2 / 4 / 8 nodes × 24 cores) on the HUIT general cpu nodes.



Figures 5c–d: 96- and 192-rank sweeps. The lower envelope keeps dropping with rank count. The vertical jumps to ~100 ms are the cluster behaving like a cluster.

At 14,400 atoms (`cpp_binned_avx512_two_pass`):

Configuration	Ranks	Time	Speedup	Efficiency
1 node × 1	1	10.95 ms	1.00×	100%
1 node × 24	24	0.79 ms	13.84×	58%
2 nodes × 24	48	0.44 ms	24.83×	52%
4 nodes × 24	96	0.29 ms	37.19×	39%
8 nodes × 24	192	0.18 ms	**59.42×**	31%

Going from 1 to 8 nodes adds another ~4× on top of single-node performance. The same trend holds across the 3k–15k atom sweet spot: 48-rank speedups land in 18–25×, 96-rank in 26–37×, 192-rank in 30–60×.

GPU Intranode MPI: structurally lower speedups + 4-rank flakiness

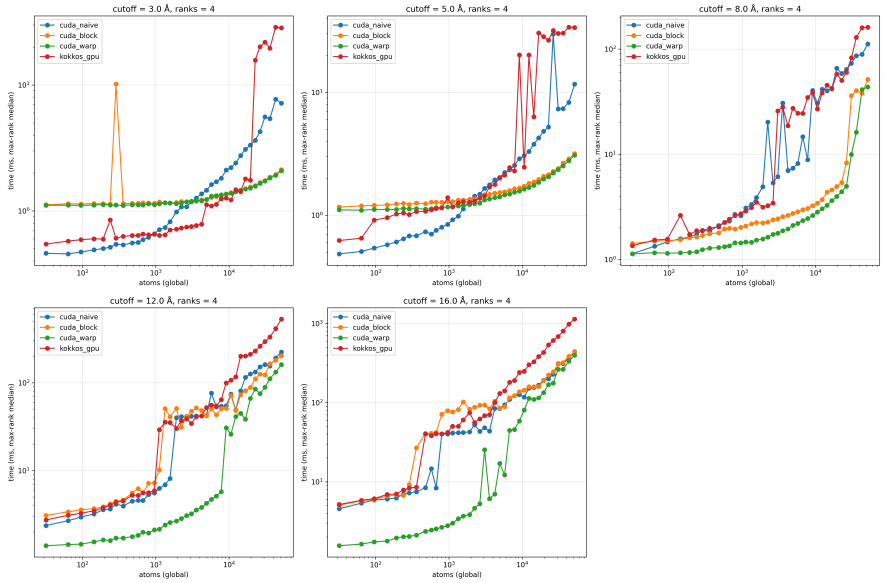


Figure 5e: 4-GPU sweep, all backends, all cutoffs. CUDA backends scale further but well below 4×; `kokkos_gpu` (red) shows reproducible vertical excursions an order of magnitude above the trend at multiple atom counts (most pronounced at cutoffs 5.0 and 8.0 Å beyond ~9k atoms).

Two things to note. First, `cuda_warp` only reaches 2.67× at 4 GPUs at the largest case — the L4 already runs the kernel in single-digit ms over 300 GB/s of HBM, so per-rank work shrinks fast and inter-GPU PCIe halo exchange becomes a comparable fraction of the step. Halo cost was a rounding error on CPU MPI; here it's half the step. Second, the `kokkos_gpu` 4-rank spikes are **deterministic** — same atom counts, same spikes, across resubmissions. The most likely cause is the same view-sizing bug that OOMs the non-MPI `kokkos_gpu` runs (preallocations of 3–4 GB per rank stacking up across 4 ranks on a shared PCIe complex), not transient noise.

A few cluster-engineering notes. CPU and GPU benchmarks live in separate venvs (`ghostly-neighbors/.venv` and `ghostly-neighbors-gpu/.venv`) because the hand-rolled AVX-512 intrinsics SIGILL on the GPU nodes' CPUs. GPU rank-to-device pinning needed a manual `.gpu_bind.sh` wrapper that sets `CUDA_VISIBLE_DEVICES=$OMPI_COMM_WORLD_LOCAL_RANK` because OpenMPI doesn't honor `--cpus-per-task` for GPU pinning here. `--gres=gpu:4 --exclusive` reserves the full 4×L4 node —

but `--exclusive` is empirically prohibited on HUIT GPU nodes, so the GPUs are contested with other jobs. That contention shows up directly as the visible spikes in the GPU MPI plots.

> GPU Multi Node: Omitted to do difficulties building and running the system. Full account recorded in the `report-full-annotated.pdf`

Framework comparison

One timing per backend at ~9,800 atoms / 5.0 Å cutoff. CPU rows use 24 OMP threads; GPU rows are single L4; MPI rows are 2 ranks. Relative-to-fastest computed across the whole table.

Backend	Device	Time	vs. fastest
cpp_binned_avx512_two_pass	CPU 24-thr	**0.83 ms**	**1.00×**
cpp_binned_avx512_omp_fused	CPU 24-thr	1.30 ms	1.56×
kokkos_cpu	CPU 24-thr	1.49 ms	1.79×
numba	CPU 24-thr	2.96 ms	3.55×
cpp_binned_avx512_two_pass	2 ranks × CPU	3.24 ms*	3.91×
cuda_block / cuda_warp	1 × L4	~3.37 ms	~4.05×
kokkos_gpu (MPI)	2 ranks × L4	3.89 ms*	4.69×
cuda_naive	1 × L4	8.42 ms	10.09×
jax	1 × L4	8.48 ms	10.17×
cpp_serial_avx512	CPU 1-thr	44.19 ms	52.97×
cpp_serial_scalar	CPU 1-thr	238.12 ms	285.43×
kokkos_gpu (non-MPI)	1 × L4	**OOM — excluded**	—
jax (MPI)	2 × L4	**not supported**	—

MPI rows measured at 9,152 atoms (closest sweep point).

Two backends have asymmetric availability. `kokkos_gpu` OOMs on a single L4 — most likely a `Kokkos::View` extents bug in the implementation — and when the OOM happens *under MPI* the dying rank never reaches the next `MPI_Allreduce`, deadlocking the surviving ranks. I excluded it from the single-GPU sweep but kept it on the MPI side, where the per-rank working set fits and it lands at 3.89 ms, comparable to the CUDA backends. JAX is the mirror image: clean on a single L4, but multi-gpu JAX setups are infamously difficult to set up so I punted.

The interesting takeaways:

- **Hand-tuned C++ on CPU still wins outright at this size.** 0.83 ms on 24 cores beats the best single-L4 CUDA kernel (3.36 ms) by ~4×. The L4's 300 GB/s is only ~2× the Xeon's 141 GB/s, and this kernel is bandwidth-bound, so the GPU's nominal compute headroom doesn't translate.
- **Productivity costs are flat on CPU.** Kokkos lands within 1.8× of hand-tuned, Numba within 3.6×.

- **The CUDA progression mirrors the CPU progression.** Naive → block/warp is ~2.5×; CPU `omp_fused → two_pass` is ~1.6×. Both speedups came from removing a non-arithmetic stall.

Conclusion

1. **Algorithm dominates.** Cell-lists give orders-of-magnitude speedup over O(N²); SIMD/threading gains are smaller in comparison.
2. **Amdahl is real.** Serial cell-list construction capped OpenMP at 1.13× until that phase was parallelized too.
3. **Autovectorization is not a programming model.** Hand-rolled AVX-512 intrinsics buy a consistent ~2× over scalar, both in O(N²) and cell-list kernels.
4. **GPU optimization is about eliminating stalls.** Barrier stalls were the bottleneck, not arithmetic. Warp shuffles eliminated 84% of stall cycles; the kernel went from "Latency Issue" to "High Throughput" with the FP64 pipeline as the new bottleneck. Same arc as the CPU `push_back` → two-pass story — different stall, same idea.
5. **MPI scales until bandwidth saturates.** Single-node 4-rank hits ~99% efficiency at the sweet spot, breaks down at ~50k atoms as DRAM saturates. Multinode buys another ~4× across 8 nodes (192 ranks: 59× over 1 rank). GPU MPI is structurally compressed because the L4 is fast enough that PCIe halo cost is half the step.
6. **Productivity vs. performance.** Numba ~28% of hand-tuned C++ at this size; Kokkos ~56% with similar simplicity. JAX trails on small problems where XLA dispatch overhead dominates.

The neighbor list is a bandwidth-bound workload at scale — arithmetic intensity stays near 0.4 FLOP/byte regardless of backend. That single property explains why every optimization in this report was about reducing data movement (compress-stores, warp shuffles, on-device prefix sums) rather than about finding more FLOPs.

Appendix: the warp-aggregated atomic pattern

The single biggest GPU win was replacing per-thread atomic appends with one atomic per warp. Phases:

```
// 1. Each lane evaluates the predicate independently.
bool found = (distance < cutoff_sq);
unsigned active = __activemask();
unsigned voters = __ballot_sync(active, found);
int n_voters    = __popc(voters);

// 2. Lowest active lane is leader; reserves n_voters consecutive slots
//    in the global output with one atomicAdd, broadcasts the base back.
int leader = __ffs(active) - 1;
int base   = 0;
if (lane == leader) base = atomicAdd(pair_count_d, n_voters);
base = __shfl_sync(active, base, leader);

// 3. Each found lane computes its unique offset by popcounting the bits
//    of `voters` below its own lane index. No collisions, no gaps.
int offset = __popc(voters & ((1u << lane) - 1));
int slot   = base + offset;
```

```
// 4. Coalesced stores — adjacent lanes write to adjacent slots.
if (found) {
    i_d[slot] = pi;
    j_d[slot] = pj;
    disp_d[3*slot]     = ddx;
    disp_d[3*slot + 1] = ddy;
    disp_d[3*slot + 2] = ddz;
}
```

Two compounding wins: one global atomic per warp (instead of up to 32) reduces L2 contention by ~10×, and the consecutive slots `base, base+1, ..., base+n_voters-1` let the memory controller coalesce 4-byte stores into wide transactions. At dense-liquid pair densities, the coalescing is usually the larger win.

Two correctness traps worth flagging: the sync mask must be `__activemask()`, not `0xFFFFFFFF` — threads that exited early (`if (pi >= n_local) continue`) are inactive, and using a full mask when inactive lanes exist is undefined behavior. And the leader must be `__ffs(active) - 1`, not lane 0, because lane 0 may itself be inactive.

References

- Allen, M. P., & Tildesley, D. J. (2017). *Computer Simulation of Liquids*. Oxford University Press.
- Verlet, L. (1967). Computer "experiments" on classical fluids. *Phys. Rev.* 159, 98.
- Kirk, D. B., & Hwu, W. W. (2022). *Programming Massively Parallel Processors* (4th ed.). Morgan Kaufmann. — Kogge-Stone / Brent-Kung scan, warp primitives.
- Amdahl (1967); Gustafson (1988); Williams, Waterman & Patterson (2009) — scaling laws and roofline.
- Intel Intrinsics Guide; Kokkos 3 (Trott et al. 2022); JAX MD; MPI 4.1; Nsight Compute; Linux `perf`.